

Homework A3 | Written Solutions

Abumere Okhihan | EECE.5821 | Due October 7, 2026

Problems 1-3: Performance and Data Hazards

1(h) Why speedup is below five

The stages are unbalanced: the 300 ps memory stage sets the clock, while faster stages have unused time. Each pipeline register adds 20 ps, making the clock 320 ps and ideal speedup $1000/320 = 3.125$. Data-hazard stalls would further reduce speedup.

Supporting calculations (Problems 1 and 2)

lw = 1000 ps; sw = 900 ps; R-type = 700 ps; beq = 600 ps.

Average = $0.25(1000) + 0.10(900) + 0.45(700) + 0.20(600) = 775$ ps.

Five-stage lw latency = $5(320) = 1600$ ps. Six-stage clock = $250 + 20 = 270$ ps;

IF is the bottleneck; speedup = $1000/270 = 3.70370$.

No-hazard cycles = $N + 4$: five instructions take 9 cycles; 100 take 104.

Cycle 5: IF I5, ID I4, EX I3, MEM I2, WB I1. Cycle 6 WB: I2, rd = x5.

Cycle 5 EX contains I3 (sw): MemWrite = 1, RegWrite = 0.

3(e) Full-forwarding pipeline diagram

	1	2	3	4	5	6	7	8	9	10	11	12
I1: lw x5	IF	ID	EX	MEM	WB							
I2: add x6		IF	ID	ID*	EX	MEM	WB					
I3: sub x8			IF	IF*	ID	EX	MEM	WB				
I4: sw x8					IF	ID	EX	MEM	WB			
I5: and x9						IF	ID	EX	MEM	WB		
I6: or x13							IF	ID	EX	MEM	WB	

* I2 waits in ID and I3 waits in IF in cycle 4; EX has a bubble.

Arrows: I1 MEM(c4) -> I2 EX(c5), x5, via MEM/WB;

I2 EX(c5) -> I3 EX(c6), x6, via EX/MEM;

I3 EX(c6) -> I4 EX(c7), x8 store data, via EX/MEM;

I5 EX(c8) -> I6 EX(c9), x9, via EX/MEM.

I3 reads x5 in ID(c5), the same cycle I1 writes WB: no forwarding needed.

With forwarding: 1 stall, $6 + 4 + 1 = 11$ cycles. Without forwarding, I2, I3,

I4, and I6 each wait 2 cycles in ID: 8 stalls, $6 + 4 + 8 = 18$ cycles.

Homework A3 | Written Solutions

Abumere Okhihan | EECE.5821 | Due October 7, 2026

Problems 4-5: Scheduling and Branch Prediction

4(b) Memory aliasing requirement

Moving a load above a store is legal only when the store cannot change the value read by that load. Separate, non-overlapping A and B arrays satisfy this condition and are the assumption used by the autograder.

Counterexample: choose $A = B + 2$ elements in a larger backing array initially $[1, 2, 4, 8, 0, 0]$. The original first store writes $B[2] = 3$. Its subsequent load of $B[2]$ reads 3, so the second store writes $A[1] = 3 - 1 = 2$. That store changes $B[3]$ to 2, and the third store writes $A[2] = 2 + 2 = 4$. The reordered schedule loads all B values first and writes $A = [3, 3, 0]$, whereas the original writes $A = [3, 2, 2]$. Thus overlapping arrays can change results.

4(a,c) Scheduling calculation

The original has load-use stalls at $lw\ x6 \rightarrow add$, $lw\ x8 \rightarrow sub$, and $lw\ x12 \rightarrow and$: 3 stalls and $10 + 4 + 3 = 17$ cycles. The submitted schedule loads all four B values first, computes the three results, and then stores them. There are no immediate load-use pairs: 0 stalls and $10 + 4 = 14$ cycles.

5(f) Why deeper pipelines benefit more from prediction

With a one-cycle penalty, prediction reduces CPI from 1.20 to 1.12; with a three-cycle penalty, it reduces CPI from 1.60 to 1.36 using the same not-taken strategy. A 90%-accurate dynamic predictor lowers the deeper pipeline CPI to 1.06, giving $1.60/1.06 = 1.50943$ speedup over stalling, because avoiding each branch penalty saves more cycles as the pipeline gets deeper.

Supporting calculations (Problem 5)

$CPI = 1 + \text{branch fraction} \times \text{penalty} \times \text{penalized fraction of branches}$.

Always stall: $1 + 0.20(1) = 1.20$; not-taken: $1 + 0.20(0.60)(1) = 1.12$.

With penalty 3: stall 1.60; not-taken 1.36; dynamic $1 + 0.20(0.10)(3) = 1.06$.

B1 executes 11 times: ten not-taken and one taken. B2 executes ten times, all taken. Total branches = 21; predict-not-taken misses 11; BTFN misses only the final B1. BTFN accuracy = $20/21 \times 100 = 95.2381\%$.

Homework A3 | Written Solutions

Abumere Okhihan | EECE.5821 | Due October 7, 2026

Graduate Problem: Forwarding Unit

(a) Complete forwarding conditions

Let $E_A = \text{EX/MEM.RegWrite} \text{ AND } (\text{EX/MEM.RegisterRd} \neq 0)$

AND $(\text{EX/MEM.RegisterRd} == \text{ID/EX.RegisterRs1})$.

Let $M_A = \text{MEM/WB.RegWrite} \text{ AND } (\text{MEM/WB.RegisterRd} \neq 0)$

AND $(\text{MEM/WB.RegisterRd} == \text{ID/EX.RegisterRs1})$.

ForwardA = 10 when E_A is true.

ForwardA = 01 when M_A AND NOT E_A is true.

ForwardA = 00 otherwise.

For ForwardB, replace RegisterRs1 with RegisterRs2 in both predicates.

RegWrite prevents forwarding from stores; $\text{Rd} \neq 0$ prevents forwarding to x0.

NOT E_A gives priority to the more recent EX/MEM value.

(b) Forwarding control values

S1: ForwardA = 10, ForwardB = 00.

S2: ForwardA = 01, ForwardB = 01 (EX/MEM is a store, so RegWrite = 0).

S3: ForwardA = 00, ForwardB = 00 (writes to x0 are ignored).

S4: ForwardA = 01, ForwardB = 10.

(c) Lecture hazard-detection results

L1: yes; the load destination x5 matches the add source x5.

L2: no; the add has MemRead = 0, and ordinary forwarding handles x5.

L3: yes; the load destination x8 matches the store data source x8.

(d) Extra forwarding path to eliminate the L3 stall

Add a path from MEM/WB writeback data to the data-memory Write data input for a store currently in MEM. For consecutive lw and sw, the loaded value is in MEM/WB while the store is in MEM, so the store data can arrive just in time.

Enable it when $\text{EX/MEM.MemWrite} = 1$, $\text{MEM/WB.RegWrite} = 1$,

$\text{MEM/WB.RegisterRd} \neq 0$, and $\text{MEM/WB.RegisterRd} == \text{EX/MEM.StoreRs2}$.

Carry the store rs2 register number through ID/EX into EX/MEM; retain the store data, MemWrite, and the WB destination, RegWrite, and selected WB value.

Also refine hazard detection: a load matching only a store rs2 need not stall;

a match on the store base rs1 must still stall, because the address is needed in EX.

(e) Why MEM/WB must not override EX/MEM

If both match, EX/MEM holds the newer result and MEM/WB holds the older one.

Example (assume $x2 = 10$, $x3 = 1$, $x4 = 4$, $x6 = 2$):

add x1, x2, x3 # older x1 = 11

sub x1, x2, x4 # newer x1 = 6

add x5, x1, x6 # must compute $6 + 2 = 8$

When the third instruction is in EX, the second is in EX/MEM and the first is

in MEM/WB. Choosing 01 instead of 10 would incorrectly compute $11 + 2 = 13$.